Database & Management System

[LAB MANUAL]

K.R. MANGALAM UNIVERSITY

SOET

(School of Engineering & Technology)

B.TECH CSE (CORE)

Submitted by:
Khushi
Roll no: **2401010089**

Submitted to:
Mansi Kajal

Experiment-1

Aim

To design an Entity–Relationship (ER) diagram for a company database system and define the relationship types between entities.

Objectives

- To identify entities and their attributes in a company database
- To represent relationships among entities
- To define relationship types such as one-to-one, one-to-many, and many-to-many

Entities and Attributes

1. Employee

Represents employees working in the company.

Attributes:

- SSN (Primary Key)
- Name
- Address
- Sex
- Salary
- SuperSSN
- DNo

2. Department

Represents departments in the company.

Attributes:

- DNo (Primary Key)
- DName
- MgrSSN
- MgrStartDate

3. Project

Represents projects handled by the company.

Attributes:

- PNo (Primary Key)
- PName
- PLocation
- DNo

4. DLocation

Represents locations of departments.

Attributes:

- DLoc
- DNo

5. Works_On

Represents the association between employees and projects.

Attributes:

- SSN
- PNo
- Hours

<u>Relationships and Relationship Types</u>

1. Works_In

Entities involved: Employee – Department

Relationship type:

- **Many-to-One (N : 1)**

Explanation:
Many employees can work in one department, but each employee works in only one department.

2. Manages

Entities involved: Employee – Department

Relationship type:

- **One-to-One (1 : 1)**

Explanation:
Each department is managed by one employee, and an employee manages at most one department.

3. Has_Location

Entities involved: Department – DLocation

Relationship type:

- **One-to-Many (1 : N)**

Explanation:
A department can have multiple locations, but each location belongs to only one department.

4. Controls / Involves

Entities involved: Department – Project

Relationship type:

- **One-to-Many (1 : N)**

Explanation:
A department can control multiple projects, but each project is controlled by only one department.

5. Works_On

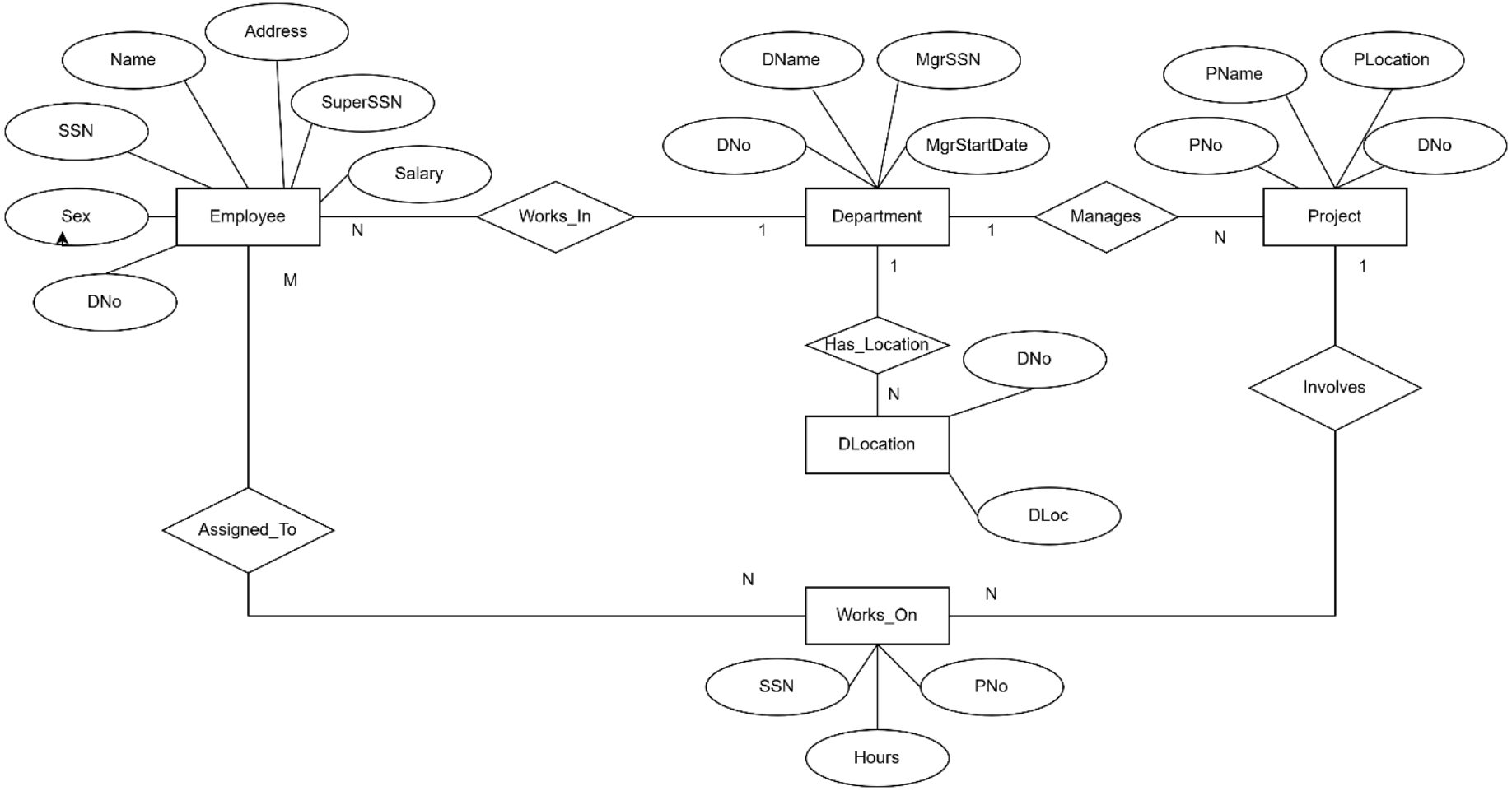
Entities involved: Employee – Project

Relationship type:

- **Many-to-Many (M : N)**

Explanation:
An employee can work on multiple projects, and a project can have multiple employees.
The attribute **Hours** indicates the number of hours an employee works on a project.

<u>Result:</u>


Address
Name
SuperSSN
SSN
Salary
Sex
Employee
DNo
N
M
Works_In
DName
MgrSSN
DNo
MgrStartDate
1
Department
1
Manages
PName
PLocation
PNo
DNo
N
Project
1
1
Has_Location
DNo
N
DLocation
DLoc
Involves
Assigned_To
N
N
Works_On
SSN
PNo
Hours

Experiment – 2

Aim

To design an Entity–Relationship (ER) diagram for a student academic management system and define the relationship types among entities.

Entities and Attributes

1. Student

Represents students enrolled in the institution.

Attributes:

- User_Name (Primary Key)
- Student_Name
- Address
- Phone_Number
- Gender

2. Semester_Section

Represents a specific section offered in a semester.

Attributes:

- SSection_ID (Primary Key)
- Semester
- Section

3. Class

Represents a class in which students are enrolled.

Attributes:

- SStudentID (Primary Key)
- User_Name

4. Subject

Represents subjects offered by the institution.

Attributes:

- Subject_Code (Primary Key)
- Title
- Credits
- Semester

5. IAMarks

Represents internal assessment marks of students.

Attributes:

- SSID
- Test1
- Test2
- Test3
- FinalIA

Relationships and Relationship Types

1. Enrolls

Entities involved: Student – Class

Relationship type:

- One-to-Many (1 : N)

Explanation:
One student can enroll in multiple classes, but each class enrollment is associated with only one student.

2. Contains

Entities involved: Semester_Section – Class

Relationship type:

- One-to-Many (1 : N)

Explanation:
A semester section can contain multiple classes, but each class belongs to only one semester section.

3. Has

Entities involved: Class – IAMarks

Relationship type:

- One-to-One (1 : 1)

Explanation:
Each class record has one set of internal assessment marks, and each IA marks record belongs to one class.

4. Assesses

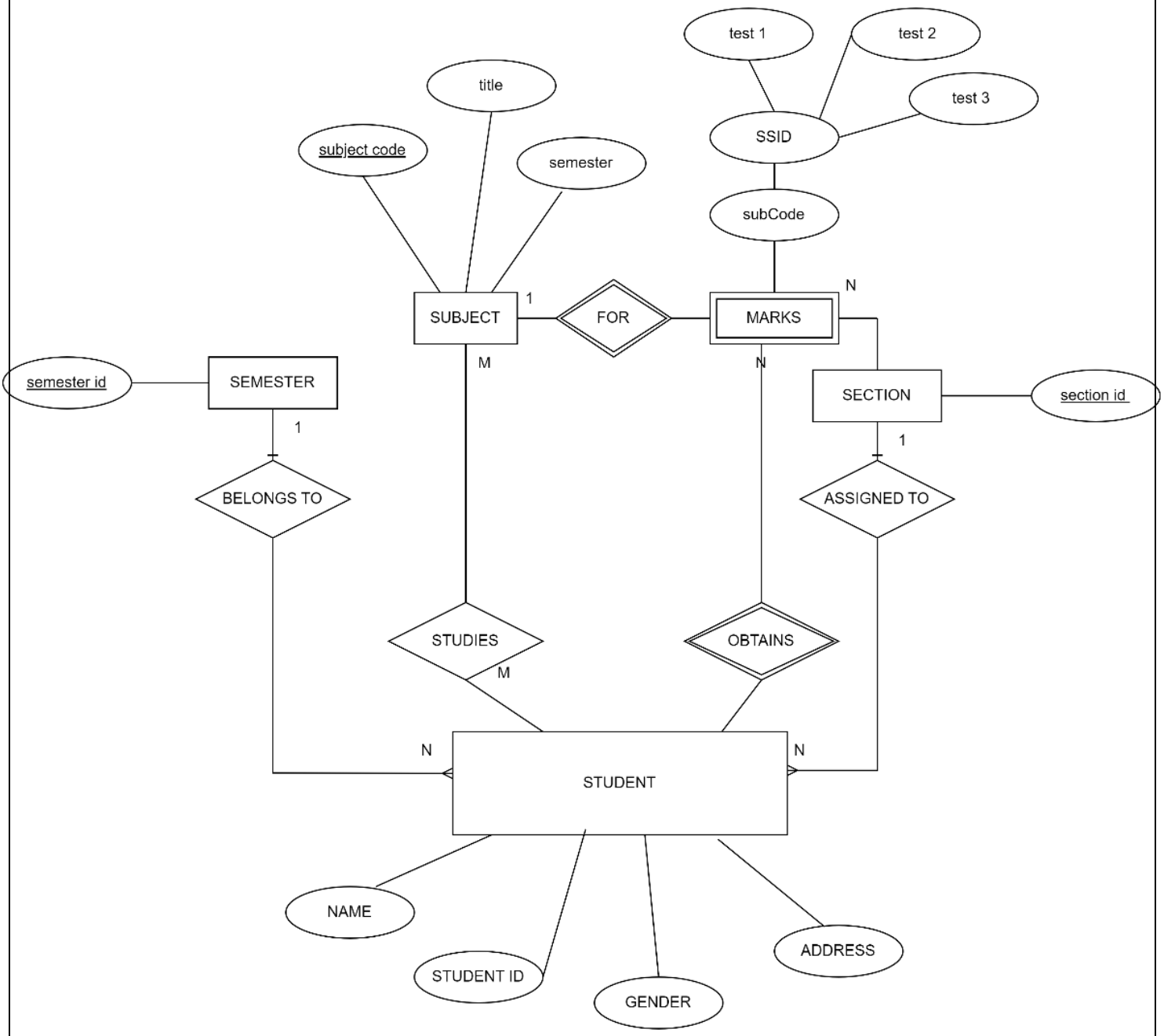
Entities involved: Subject – IAMarks

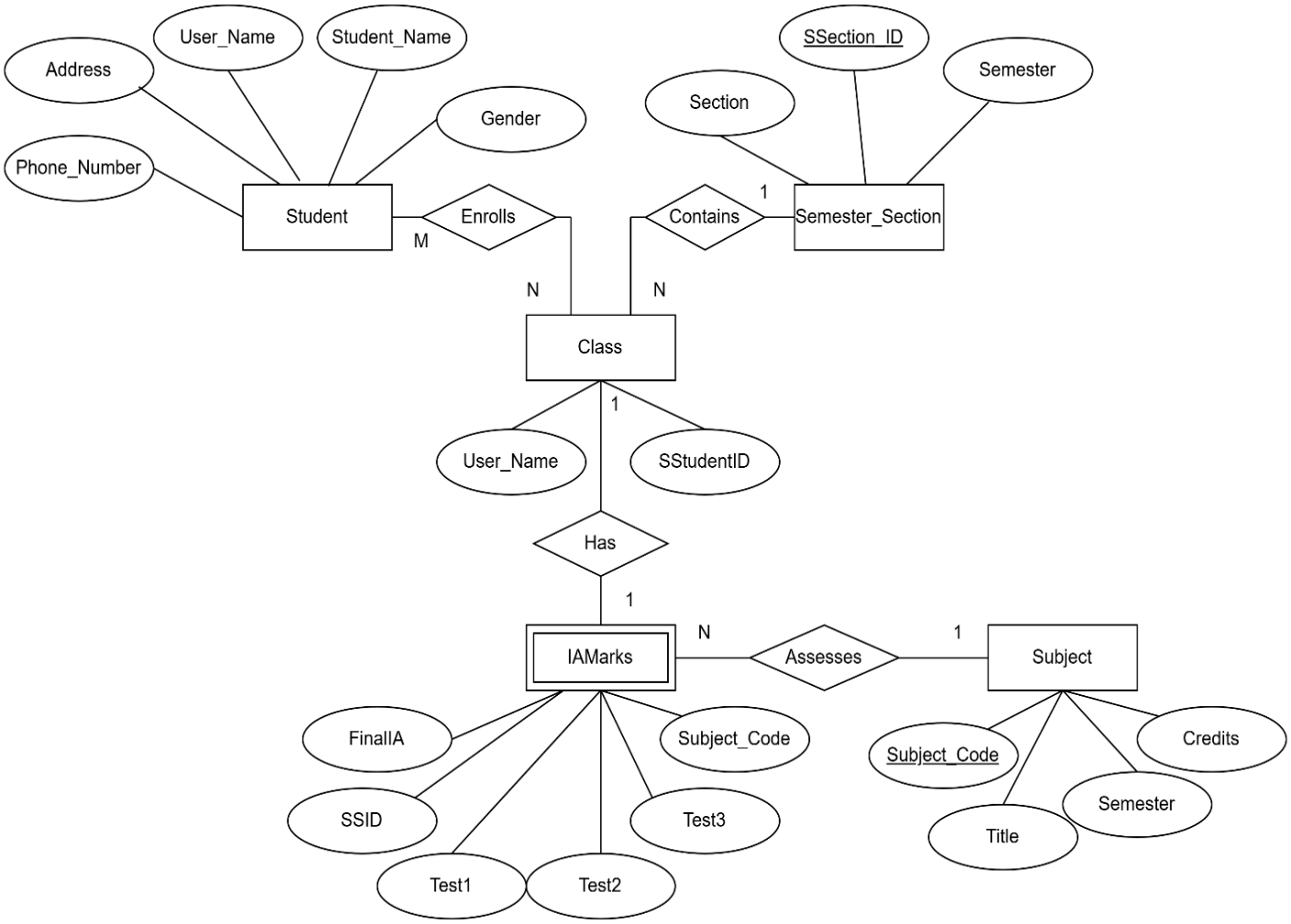
Relationship type:

- One-to-Many (1 : N)

Explanation:
A subject can assess many IA marks records for different students, but each IA marks record is related to only one subject.

Result:


test 1
test 2
test 3
title
SSID
subject code
semester
subCode
SUBJECT
1
FOR
MARKS
N
M
N
semester id
SEMESTER
SECTION
section id
1
1
BELONGS TO
ASSIGNED TO
STUDIES
OBTAINS
M
N
N
STUDENT
NAME
STUDENT ID
GENDER
ADDRESS


User_Name
Student_Name
Address
Gender
Phone_Number
Student
Enrolls
M
N
SSection_ID
Semester
Section
Contains
1
Semester_Section
N
Class
1
User_Name
SStudentID
Has
1
IAMarks
N
Assesses
1
Subject
FinalIA
Subject_Code
SSID
Test3
Test1
Test2
Subject_Code
Credits
Semester
Title

Experiment – 3

Create Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

2. Insert around 10 records in each of the tables.

3. List all the bills for the current date with the customer names and item numbers.

4. List the total Bill details with the quantity sold, price of the item and the final amount.

5. List the details of the customer who have bought a product which has a price>200.

6. Give a count of how many products have been bought by each customer

7. Give a list of products bought by a customer having cust_id as 5.

8. List the item details which are sold as of today.

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

10. Create a view which lists the daily sales date wise for the last one week

---

1-Create DATABASE

```
CREATE DATABASE salesdb;
        USE salesdb;
```

1.2 - Create the Tables with Integrity Constraints

```
CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
```

```
    cust_name VARCHAR(100) NOT NULL
);



CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    price INT CHECK (price > 0)
);

CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT CHECK (qty_sold > 0),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

2- Insert Around 10 Records in Each Table

```
INSERT INTO Customer VALUES
(1, 'Amit'),
 (2, 'Khushi'),
 (3, 'Ravi'),
 (4, 'Neha'),
 (5, 'Arjun'),
(6, 'Meera'),
(7, 'Sohan'),
```

(8, 'Priya'),

(9, 'Vikas'),

(10, 'Anita');

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

cust_id	cust_name
1	Amit
2	Khushi
3	Ravi
4	Neha
5	Arjun
6	Meera
7	Sohan
8	Priya
9	Vikas
10	Anita
NULL	NULL

customer 1

INSERT INTO Item VALUES

(101, 'Laptop', 55000),

(102, 'Mouse', 500),

(103, 'Keyboard', 1200),

(104, 'Monitor', 8000),

(105, 'Phone', 25000),

(106, 'Charger', 700),

(107, 'Headphones', 2200),

(108, 'Tablet', 18000),

(109, 'Camera', 30000),

(110, 'Speaker', 3500);

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

item_id	item_name	price
101	Laptop	55000
102	Mouse	500
103	Keyboard	1200
104	Monitor	8000
105	Phone	25000
106	Charger	700
107	Headphones	2200
108	Tablet	18000
109	Camera	30000
110	Speaker	3500
NULL	NULL	NULL

item 1

INSERT INTO Sale VALUES

(1001, CURDATE(), 1, 101, 1),

(1002, CURDATE(), 2, 102, 2),

(1003, CURDATE(), 3, 105, 1),

(1004, CURDATE(), 4, 107, 3),

(1005, CURDATE(), 5, 109, 1),

(1006, CURDATE(), 6, 110, 2),

(1007, CURDATE(), 7, 104, 1),

(1008, CURDATE(), 8, 103, 2),

(1009, CURDATE(), 9, 108, 1),

(1010, CURDATE(), 10, 106, 1);

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	qty_sold
1001	2026-02-18	1	101	1
1002	2026-02-18	2	102	2
1003	2026-02-18	3	105	1
1004	2026-02-18	4	107	3
1005	2026-02-18	5	109	1
1006	2026-02-18	6	110	2
1007	2026-02-18	7	104	1
1008	2026-02-18	8	103	2
1009	2026-02-18	9	108	1
1010	2026-02-18	10	106	1
NULL	NULL	NULL	NULL	NULL

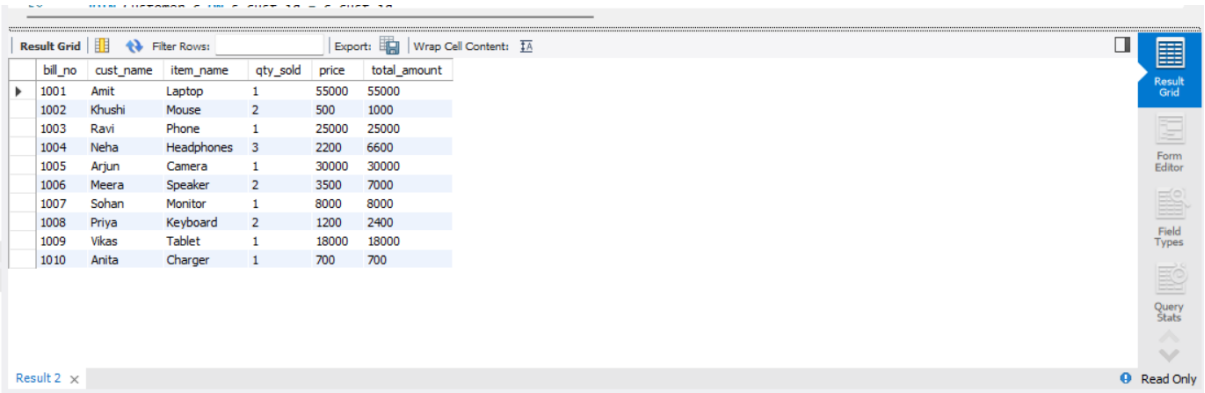
sale 1

3. List All Bills for Current Date with Customer Names and Item Numbers

```
SELECT s.bill_no, s.bill_date, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

4. List Total Bill Details (qty, price, final amount

```
SELECT s.bill_no, c.cust_name, i.item_name, s.qty_sold, i.price,
    (s.qty_sold * i.price) AS total_amount
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id;
```



bill_no	cust_name	item_name	qty_sold	price	total_amount
1001	Amit	Laptop	1	55000	55000
1002	Khushi	Mouse	2	500	1000
1003	Ravi	Phone	1	25000	25000
1004	Neha	Headphones	3	2200	6600
1005	Arjun	Camera	1	30000	30000
1006	Meera	Speaker	2	3500	7000
1007	Sohan	Monitor	1	8000	8000
1008	Priya	Keyboard	2	1200	2400
1009	Vikas	Tablet	1	18000	18000
1010	Anita	Charger	1	700	700

5.Customers Who Bought Products with Price > 200

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name
1	Amit
2	Khushi
8	Priya
7	Sohan
3	Ravi
10	Anita
4	Neha
9	Vikas
5	Arjun
6	Meera

Result 3 × Read Only

6. Count of Products Bought by Each Customer

SELECT c.cust_id, c.cust_name, COUNT(s.item_id) AS products_bought

FROM Sale s

JOIN Customer c ON s.cust_id = c.cust_id

GROUP BY c.cust_id, c.cust_name;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name	products_bought
1	Amit	1
2	Khushi	1
3	Ravi	1
4	Neha	1
5	Arjun	1
6	Meera	1
7	Sohan	1
8	Priya	1
9	Vikas	1
10	Anita	1

Result 4 × Read Only

7. Products Bought by Customer with cust_id = 5

SELECT i.item_id, i.item_name

FROM Sale s

JOIN Item i ON s.item_id = i.item_id

WHERE s.cust_id = 5;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_id	item_name
109	Camera

Result 5 × Read Only

8. Item Details Sold Today

```
SELECT DISTINCT i.item_id, i.item_name, i.price
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date = CURDATE()
```

9. Create a View for Bill Details

```
CREATE VIEW BillDetails AS
SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id,
    i.price, s.qty_sold, (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-18	1	101	55000	1	55000
1002	2026-02-18	2	102	500	2	1000
1003	2026-02-18	3	105	25000	1	25000
1004	2026-02-18	4	107	2200	3	6600
1005	2026-02-18	5	109	30000	1	30000
1006	2026-02-18	6	110	3500	2	7000
1007	2026-02-18	7	104	8000	1	8000
1008	2026-02-18	8	103	1200	2	2400
1009	2026-02-18	9	108	18000	1	18000
1010	2026-02-18	10	106	700	1	700

billdetails 1 ×

10. Create a View for Daily Sales (Last 1 Week)

```
CREATE VIEW WeeklySales AS
SELECT s.bill_date, SUM(s.qty_sold * i.price) AS daily_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
```